NEURON MODEL:

→ Electric signals
Dendrites (input receive)
Cell body (Soma) (to process input)
Axon (to carry input)
Synapses (to pass - connecting link)

December 20, 2017 Books →

→ **Neural Networks**: information processing paradigm inspired by biological nervous system, such as brain.

Structure
- large no. of highly interconnected processing elements working together (neurons)
- can learn from experience (by examples)

→ **Artificial Neural Networks**
- data processing system consisting of a large no of highly interconnected processing elements in an architecture inspired by the structure of ~~cerebral cortex of brain~~ cerebral cortex of brain

Tsoukalas & Uhrig (1997)

- Neurons →
- Dendrites → rec signals from other Neurons
- Axons → passes info via axon to other Neuron
- Synapses → weights (change in weights → learning occurs)
- Soma/Sum → Soma sums incoming information
- after sufficient info the cell fires via Axons to other neurons.
- threshold value is the point which decides firing of a Neuron.
- Neurons only cell in body with processing abilities.

→ is an info processing system having certain performance characteristics with biological nets

→ Key features of NN:
1. processing elements receive many signals
2. signals are modified by ~~rec~~ the signals
3. processing units sums the weighted sums.
4. Under appropriate circumstances (sufficient inp.) the neurons transmits a signal ~~op~~ (firing of a neuron).
5. Output from particular neuron may be passed to many neurons.

December 21, 2017

→ Strength of connection b/w the neurons is stored as a weight value for the specific connection.

→ learning the solution to problem = changing connecting weights

→ the min and max values of any problems is optimization.

→ ANN have been developed as generalization of Mathematical models of neural biology.

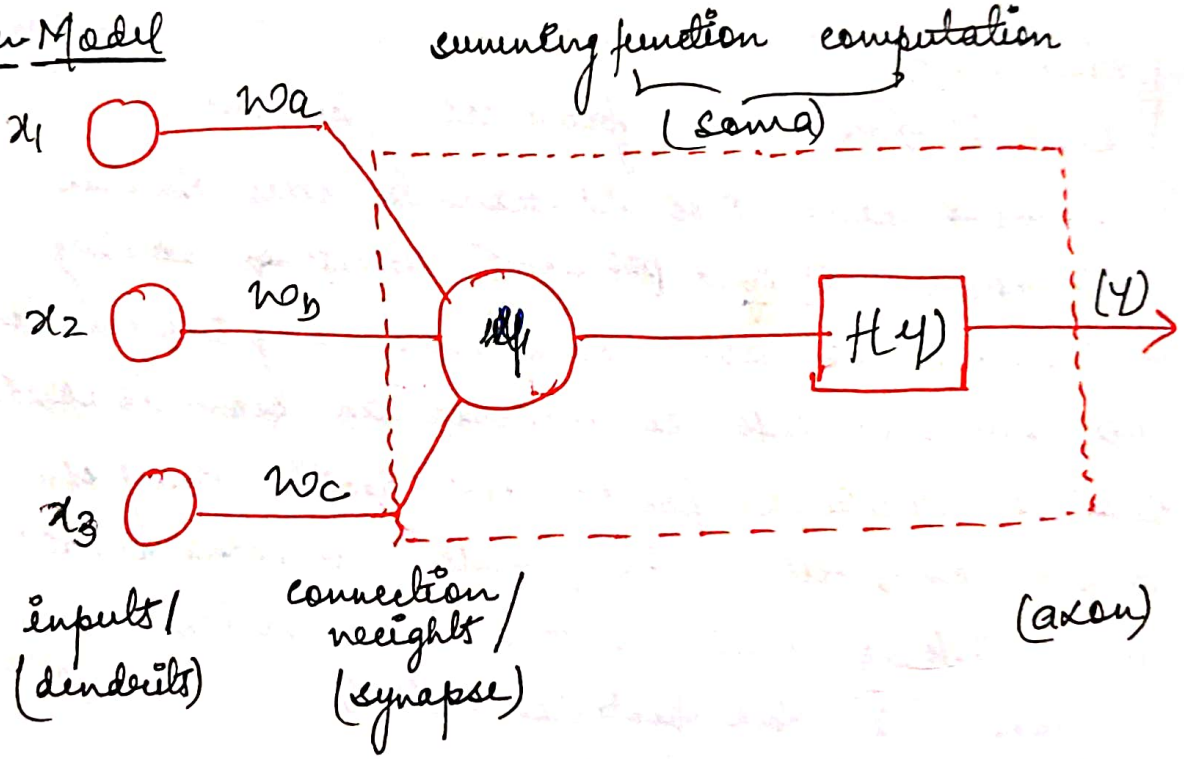
① Info processing is b/w simple elements called Neurons.

② Signals are passed b/w neurons over connection link.

③ Each connection link has weights

④ Each neuron applies an activation function.

→ Neuron Model



→ Neural nets consists a large no of simple processing unit Neuron / Unit / Cell / Nodes.

→ Communication link Axons with associated weight.

→ Weights represent information being used by the net to solve the problem.

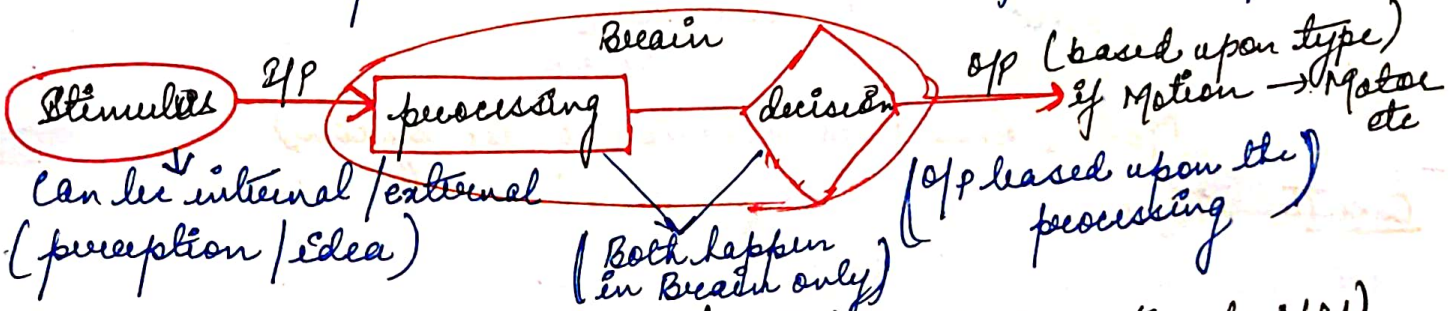
→ Weight can be 0, so that i/p has no existance.

→ Each neuron has internal state (instance), called its activation or activity level, which is a fn of i/ps it has Rx. Typically, a neuron sends its data activation as a signal to several neurons.

→ Neurons can send only one signal at a time, although that signal is broadcast to several other neurons.
→ NN are configured for pattern recognition, data classification through learning.
→ In a biological system, learning involves adjustment to the synapse connections b/w neurons.
→ Same for ANN also. (artificial Neural Networks)

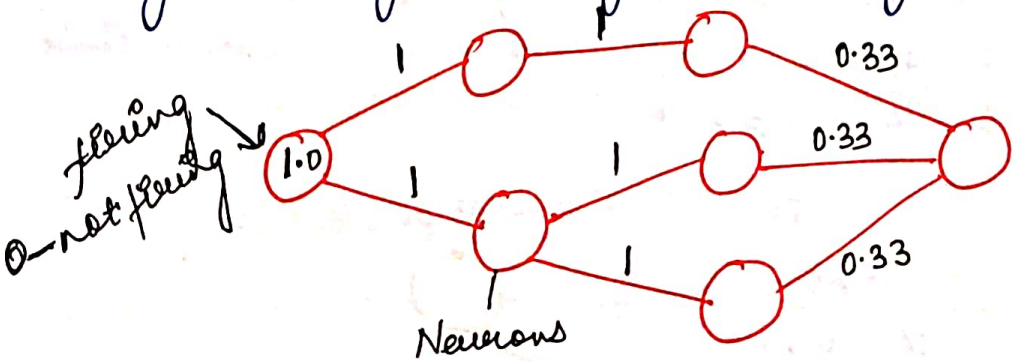
January 3, 2018 → **HOW BRAIN WORKS? (Working of Neuron)**

→ Brain is complex but it is divided into a few basic operations.



NOTE: Creating a Neural Network (Article → Making a Simple NN)
(Becoming Human - artificial intelligence) Magazine)

→ Neuron is the basic unit of computation in the brain; it R✗ and integrates chemical signals from other neurons.
→ all are running in parallel (parallel processing → Supercomp etc)
Eg: Hadoop, BIG data, data science (parallel computing eg)
→ One neuron is getting its data from its predecessor.
→ Neurons are in ready state (just activation is required)
→ Gradual growth & gradual decay (not immediate except fire)



weight can be b/w (-1 to +1)

• connection represent forward movement of info. (left-right)

→ the weights are (b/w (+(-1)) they are manipulated to alter the changed of the Neuron b/w them if fire → learning (if Not → No learn
→ The change in weights either makes Neuron fire or not.
→ Learning is dependent on the modification of weights.

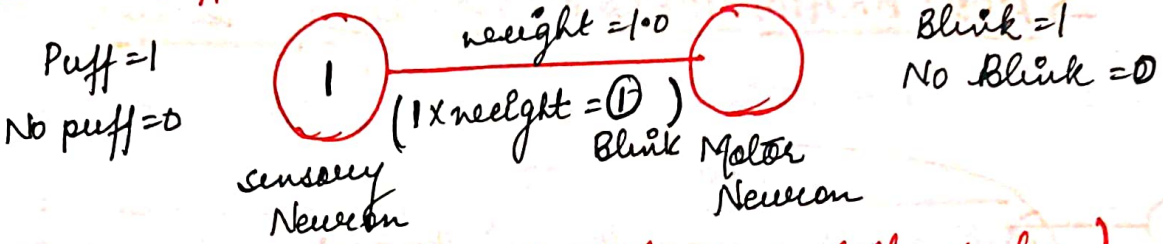
→ Increasing weights ⇒ Increasing performance

→ A Test Bunny (To understand how weights affect Neurons)

→ when subjected to a harmless puff of air, bunnies like human usually blink.

→ this behaviour (reaction to an action) can be modeled into a graph:

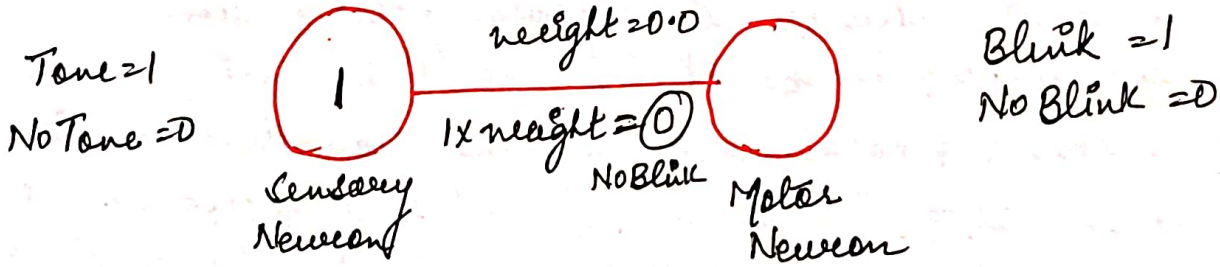
Case I Puff



Model (represents behaviour of the system)

Case II Tone

→ Eyes do not blink in this case.

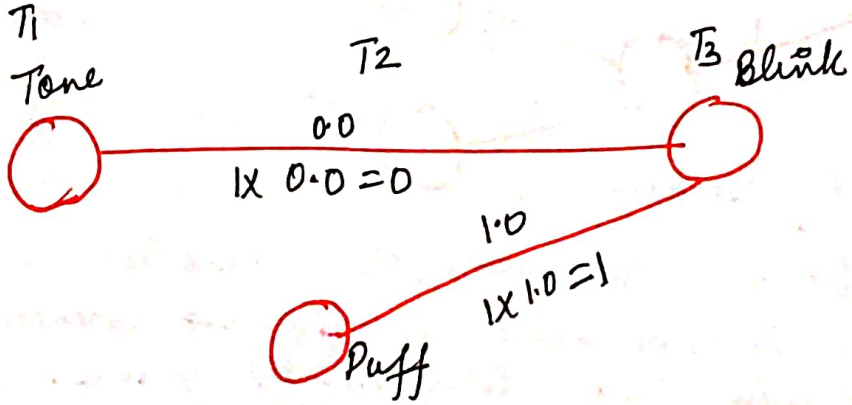


Case III Training (Mix-Tone/Puff)

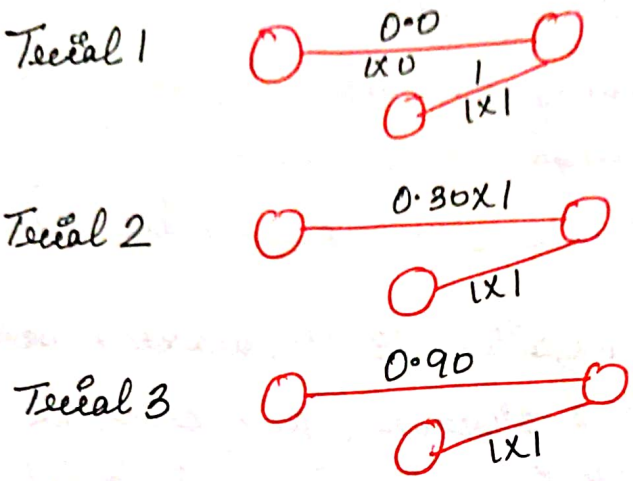
→ It is not necessary that training always leads to learning.

→ each of the events happen at diff epoch or moments of time.

Tone Puff (T1) —— (T2) Puff ——→ (T3) Blink

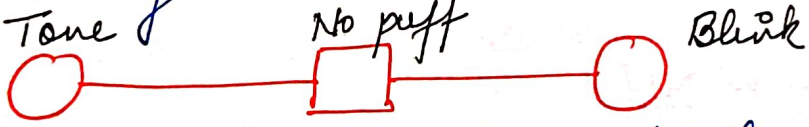


→ Learning while a complex behaviours can be simplified for the time being as the incremental change in b/w connected neuron is Neural Network through time.



Bunny trials

→ after training (trials)

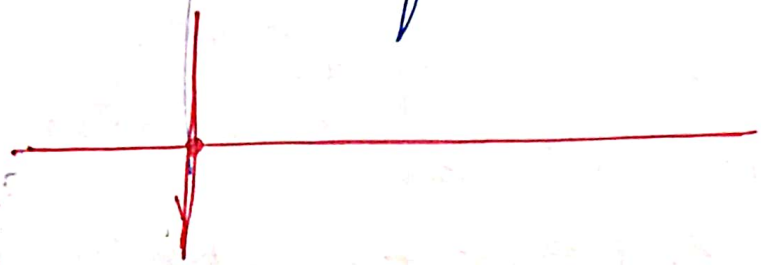


→ the bunny is trained that initially tone then air puff so even if after tone there is no puff the bunny blinks due to psychological ~~aspect~~ aspect.

→ Activation function:

① Thresholding function: (Mc-Culloch Pitts Models)

→ Simplest activation function is thresholding f?

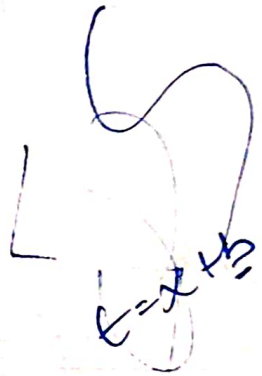


Make value of Y as 0 or 1

$$Y = \theta(x+b)$$
$$= \theta(t)$$

(induced field)

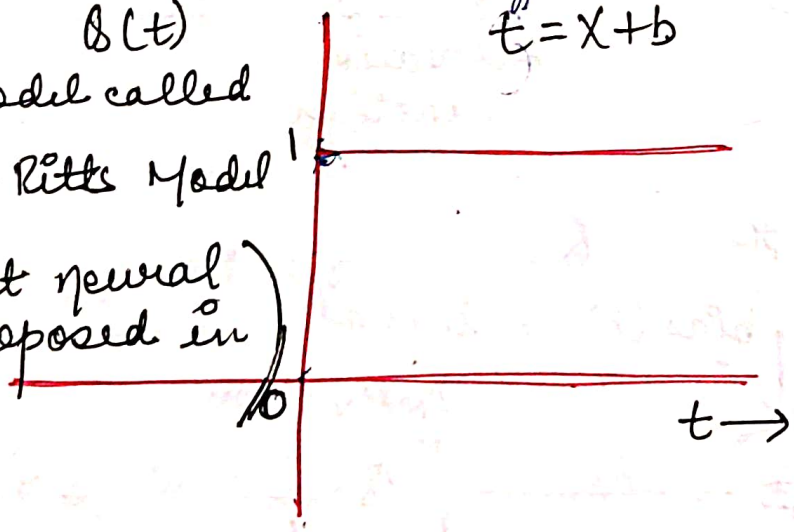
$t = x + b$



$\theta(t)$

popular Model called Mc Culloch-Pitts Model

(very first neural model proposed in 1943



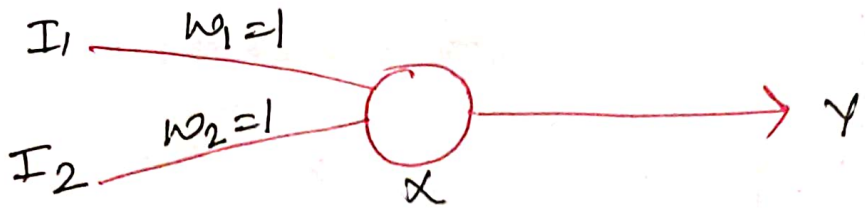
$t = x + b$

$$\theta(t) = \begin{cases} 1 & t \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

$t = x + b$.

induced field (introduced by external variable 'b')

⑤ OR Gate



Assume weights are equal and 1

$$Y = \theta(x) = \frac{1}{1+e^{-x}}$$

I_1	I_2	Y
0	0	0
0	1	1
1	0	1
1	1	1

$X = I_1 + I_2$

$I_1=0, I_2=0$

$Y = \phi(x)$

$x = 0$

$Y = \frac{1}{1+1}$ $\quad Y=1/2$ $\quad Y = 0.5$

$X = 1+1 = 2 \quad (x=1)$

$Y = \phi(2)$

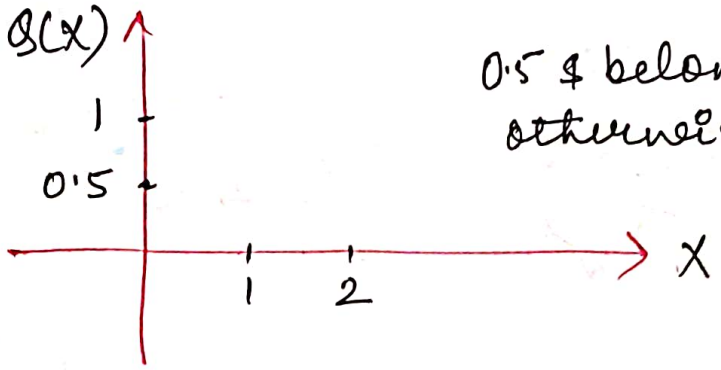
$Y = \frac{1}{1+e^{-2}}$

$Y = \frac{1}{1+\frac{1}{e^2}}$

$Y = \frac{e^2}{e^2+1}$

$Y = \frac{2.71^2}{(2.71)^2+1}$

$Y = \frac{9}{9+1} \Rightarrow Y = \frac{9}{10} \Rightarrow$ **$Y = 0.88 \approx 1$**



0.5 & below = 0
otherwise = 1

$I_1 = 0, \; I_2 = 1$

$X = I_1 + I_2$

$X = 0 + 1 = 1$

$Y = \phi(X) = \frac{1}{1+e^{-X}} = \frac{1}{1+e^{-1}}$

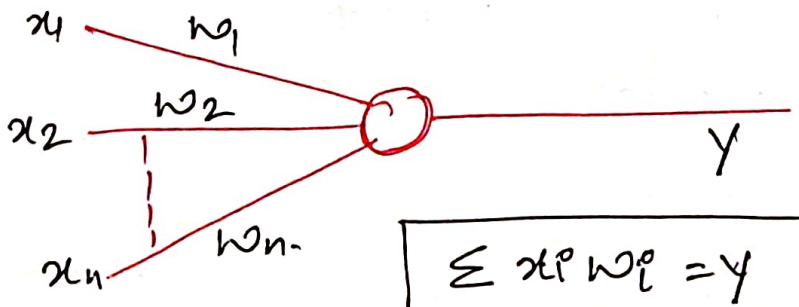
$= 0.73$

$I_1 = 1, \; I_2 = 0, \; X = 1$

$\boxed{Y = 0.73 \approx 1}$

July 19, 2017

→ Mc-COLLOCH PITTS MODEL



$\boxed{\sum x_i w_i = y}$

AND

x_1	x_2	Y
1	1	1
1	0	0
0	1	0
0	0	0

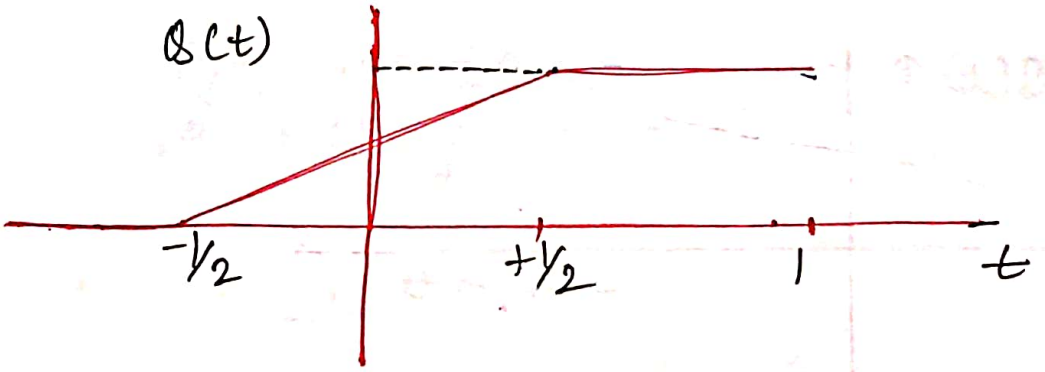
Activation function = $\phi(x)$

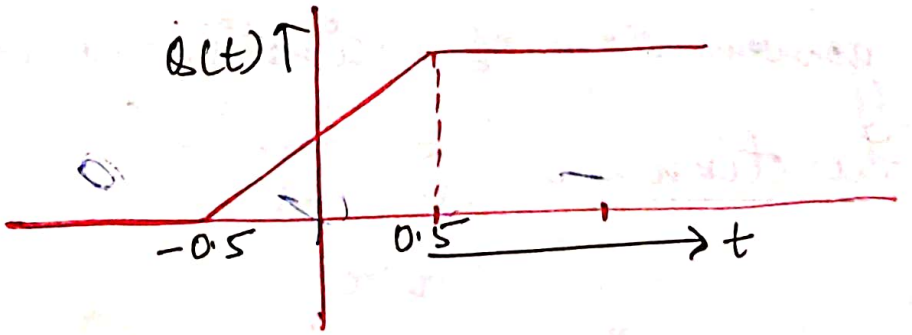
→ Adder function is a linear combination of Inputs & weights proposed in 1943.

→ With the help of Mc-Culloch Pitts Model NAND, AND, OR gate were made

② Piecewise linear function :

$$\phi(t) = \begin{cases} 1 & t \geq +1/2 \\ t & +1/2 < t < -1/2 \\ 0 & t \leq -1/2 \end{cases}$$





$t = \frac{1}{4}$, $\phi(t) = 1/4$

$t = \frac{-1}{4}$, $\phi(t) = -1/4$

$t = \frac{1}{8}$, $\phi(t) = \frac{1}{8}$

$t = \frac{-1}{8}$, $\phi(t) = \frac{1}{8}$

③ Sigmoid Function:

→ S-shaped curve.

$$\delta(t) = \frac{1}{t e^{-at}}$$

→ to plot this graph we need two values (a & t)

$$\delta(x) = \frac{e^{5}}{e^{5+1}}$$

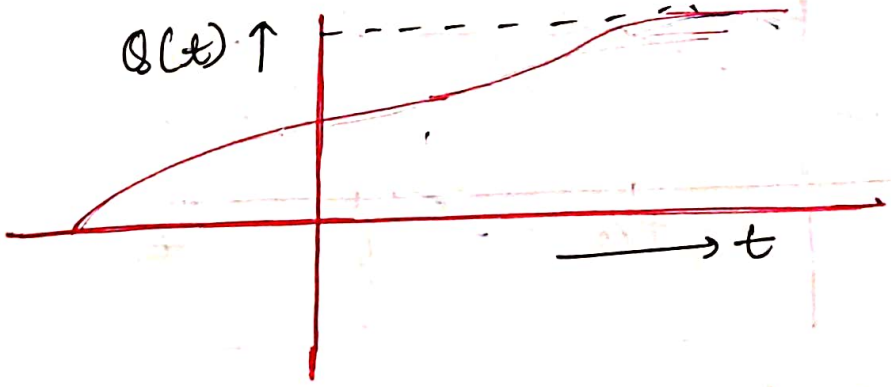
$$= e^{-1000}$$

$$= e^{1/1000}$$

$$\approx 0$$

$\delta(x) \to 1$

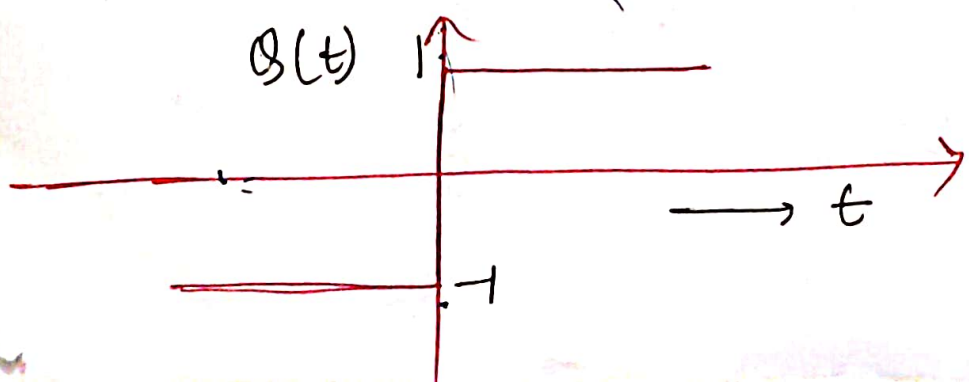
asymptotic curve.

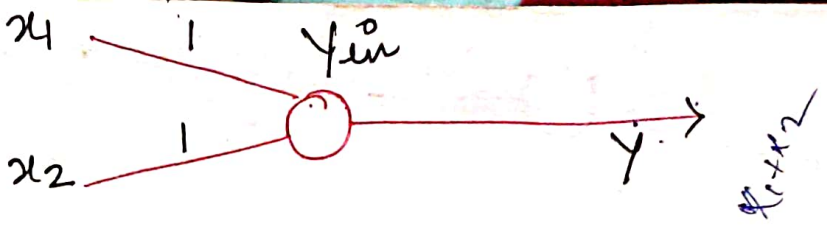


→ a will govern slope of activation function.

④ Signum Function:

$$\delta(t) = \begin{cases} 1 & t > 0 \\ 0 & t = 0 \\ -1 & t < 0 \end{cases}$$





$$y_{in} = \sum w_i x_i$$
$$= \sum x_i$$
$$= x_1 + x_2$$

① Now, $x_1 = 1, x_2 = 1$

$$y_{in} = x_1 + x_2$$
$$= 1 + 1 = 2$$

$$y = f(y_{in}) = 2 \geq 2^{T}, \quad y = 1$$

$$y = \begin{cases} 1 & y_{in} \geq 2 \\ 0 & y_{in} < 2 \end{cases}$$

② $x_1 = 1, x_2 = 0$

$$y_{in} = x_1 + x_2 = 1 < 2$$
$$y = 0$$

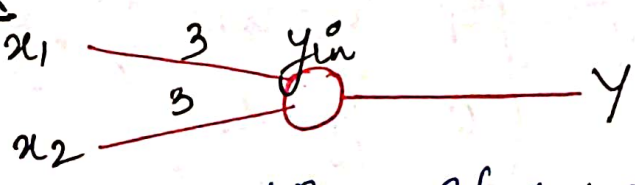
③ $x_1 = 0, x_2 = 0$

$$y_{in} = x_1 + x_2 = 0$$
$$y = 0$$

④ $x_1 = 0, x_2 = 0$

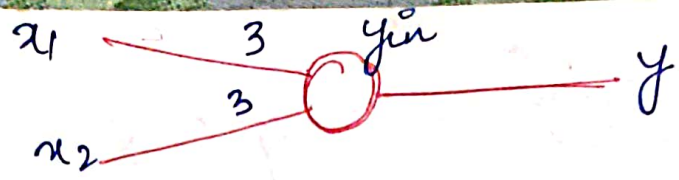
$$y_{in} = x_1 + x_2 = 0$$
$$y = 0$$

OR



$$y_{in} = 3(x_1 + x_2)$$

x_1	x_2	y	y_{in}
1	1	1	6
1	0	1	3
0	1	1	3
0	0	0	0



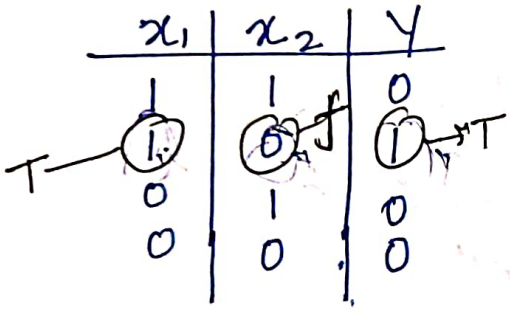
$x_1 = 0, \; x_2 = 0$

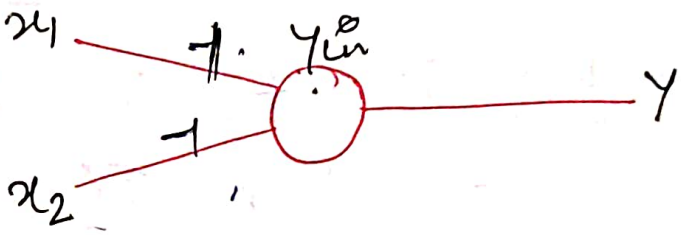
$y_{in} = x_1 + x_2 = 0$

$$Y = \begin{cases} 1 & y \geq 3_{-1} \\ 0 & Y < 3 \end{cases}$$

AND-NOT

x_1	x_2	Y
1	1	0
1	0	1
0	1	0
0	0	0





$y_{in} = x_1 - x_2$

(i) $x_1 = 1 = x_2$

$y_{in} = 0$

- $x_1 = 1, \; x_2 = 0$ $\quad y_{in} = 1$

$x_1 = 0, \; x_2 = 1$ $\quad y_{in} = 1$

$x_1 = 0, \; x_2 = 0$ $\quad y_{in} = 0$

Deterministic Model

$$Y = \begin{cases} 1 & y_{in} \geq 1 \\ 0 & y_{in} < 1 \\ \text{otherwise} \end{cases}$$

} we are assuming particular threshold

→ KNOWLEDGE Representation

1) **Data**: facts / records → collection of symbols, numbers, etc also called Raw Bits.

2) **Information**: processed data (categorised in numbers, vowels alphabets etc) classifying the data.

3) **Knowledge**: understanding, meaningful information, contextual information, learning.

→ **Classification**:

→ give you the understanding of the available information

→ **Pattern classification**:

→ Vowels and consonants

→ **2 Classification**: Binary level & Multi level
(either 0/1) (Many classes)

Say: M.Tech → Students → data
Smard card Id → Information (classified)

→ classification 0/1 Human & Non-Human → Binary Classification

→ Humans can Identify but machine cannot. So process data and Information teach system to differentiate between human & Non-human. i.e make machine Intelligent.

→ TO MATCH

Calculate difference with Smart card Number/Id
- 0 → MTech
- 1 → Non-MTech

Eg → Binary classification (Human & Non-Human)

Human

⇒ calculate the edge (Boundary) in diff. positions:
1) while standing
2) while sitting
3) side face pete

→ calculate → store in Array/Matrix

Non-Human

→ edge for all other entities other than humans.
say: cars, cattle

→ Mixed data (Human + Non Human) edge features
if Euclidian distance ≈ 0 then Human else not.

→ categories of feature Vectors is available, we can choose amongst them.

→ NN is used for pattern classification, whether it is a 2 class or Multiclass problem.

→ EMOTIONS/EXPRESSION FACIAL:

→ Diff. emotion Happy Sad Shock Fear Digest above are 5 classes so multi-class classification:
if → else → elseif → elseif → elseif ... so on

→ each class needs sample data

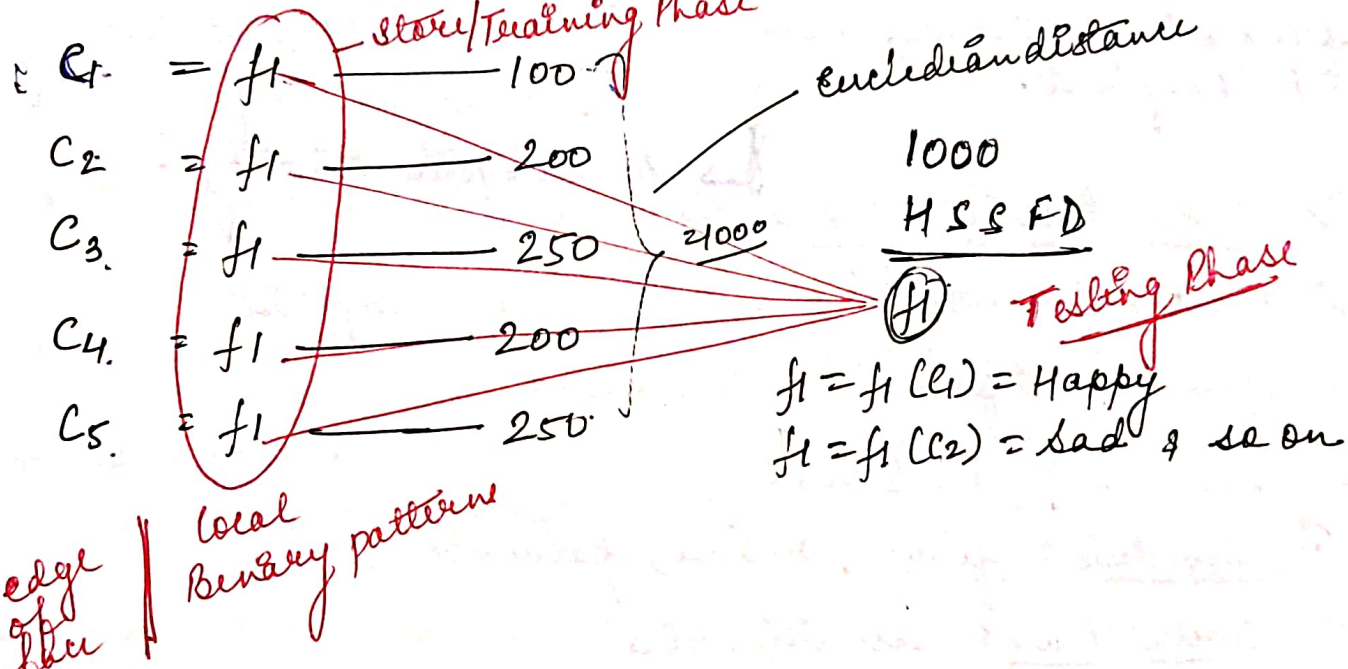
① Happy: front, up, down, side etc --- say 10

② Sad: front, up, down, side etc -- and so on.

NOTE: Above data are for only single person.

Expressions → Happy Sad Shock Fear disgust

data → 10 10 10 10 10



edge of face | Local Binary patterns

→ we have 1000 images of Happy / sad --- disgust

→ compare and calculate the distance.

→ deep learning: is a kind of NN.

→ Knowledge Representation:

→ Fischler and Firschein in 1987.

→ Knowledge refers to stored information of models used by a person or machine to interpret, predict & appropriately respond to the outside world.

→ this know should be supported by some prior information

→ Knowledge representation is required to classify data or to design a particular task.

Labelled data

→ system information has information to human.

→ information available in system to identify is labelled data

→ predefined objectives, desired o/p is defined.

Unlabelled data

→ information related to other things other than Humans.

→ grouping things on basis of figure. ie

say: cycles → two wheels regardless of the Manufacturing Companies

→ let the system identify.

→ **Characteristics of Knowledge:**

a) What information is actually made explicit.

b) How the information is encoded.

→ Real world problems:

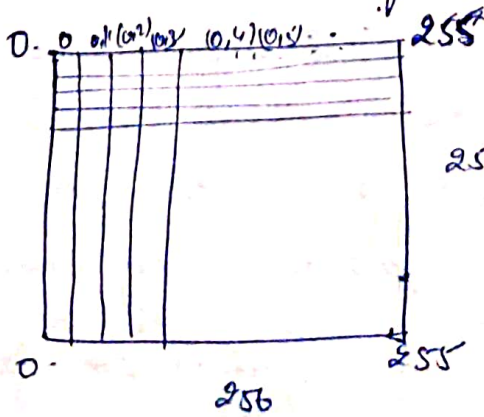
① **Biometric**: finger, retina, palm etc.

② **Surveillance**: (security, video, activity Recognition abnormal activities etc.

July 28, 2017

→ Gray scale / ~~Black and white~~ photo $- 2^8 = 256$. (0 – 255)
(Black (no info.) and White (info))

→ colour photo $- 2^{24}$ number of colours



→ Invariance: No matter how the image rotates system should read and identify the value.

→ Rotational:

Say: in Biometric System, thumb impression is registered, either thumb is placed vertically, or perpendicularly. Still system identifies so system is invariant with rotation.

→ Scaling: either expand or contract any image matrix by multiplying it by any number.

(256 x 256) ×2 516 X 516.

(256 x 256) X 1/2

→ Translation: movement of object; move, shift still object should be identified.

Eg palm recognition, finger, thumb, iris, face, ear, character

→ No matter how the character is written system should be able to recognise then only system will be Intelligent else not. Invariance is very important for system designing.

→ to classify neural classifier is the Neural Network/ Fuzzy Logic are classifiers only. also called Pattern Classifier.

→ there are three phases of system design:

1. Training
2. Testing
3. Validation (Error Management)

→ Problems after System Designing:

a) What are the features to select/consider?

→ Better classifier is one with minimum no. of inputs, and covers maximum characteristics and gives accurate results.

July 29, 2017

→ Knowledge Representation:

→ required to classify the data & to design the system

→ feature based learning.

→ Histogram: No of times n element is repeated. Graphical representation of data.

→ frequency of any element (Repratation of Element) Bar Gr
→ data will be Binarized (Values will be either 0/1) so edge will be either white / Black.
→ Local Binary pattern (Gray)

⋮

n features

all ~~all~~ the above are called feature descriptors for NN.

→ Knowledge can be representation can be done by:
① Prior information
② Measurement observations.

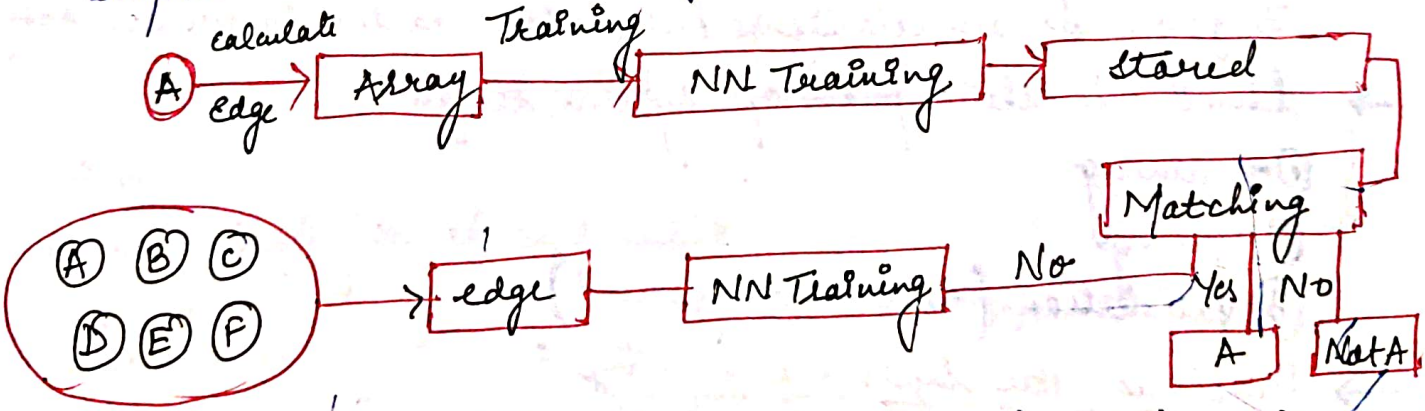
→ NN should be trained & tested for Know. Rep. to find the kind of inputs. processing of datta

8	9	3	7
18	20	9	5
12	8	11	5
9	7	3	9

4×4

Normalize the data
Greatest value in Matrix = 20
8/20, 9/20, 3/20, 7/20 ... 9/20 ... all values b/w (0~1)

→ feature selection should be optimum as selection of features and then processing the data to get the desired output.



→ length of feature Vector & Accuracy. there should be a trade-of

$$\text{Accuracy} = \frac{\text{No of identified}}{\text{Total i/p}} \times 100$$

$$\text{Say } A = \frac{700}{1000} \times 100 \Rightarrow 70\%$$

① Prior Information: available information

② observation Measurements: performing operations on data & observing

→ Mixed data passed (Natural → Trees, Animals, Unnatural – Chair table) system will itself how to group data on basis of features called Unlabelled data. all bicycles under one group, all chairs under one group called as Clustering. used in Training / Testing. It is called Unsupervised Learning.

→ Training the system to get the desired result is Labelled data called Supervised Learning.

→ Classification of Types:

① Binary:

② Multiclassification:

Eg: Human Activities:

Running	Sitting	Boxing	Walking	Jumping
10	10	10	10	10

→ 5 classes
→ labelled data
→ initially we need sample data
→ features based learning
→ train the system

Say 1000 ip/p ⟹ R→110, S→200, B→300, W→300, J→90. this is Testing. (data can be passed either in one time or 2 times 500/500 & so on)

→ increasing 100, 100, 100, 100, 100 & equal no of 500 so this type of system is inefficient system.

① Supervised learning:

- a strategy that involves a teacher that is smarter than n/w itself.
- say: Teacher shows n/w a bunch of faces. already the associated names are known to the teacher.
  N/w guesses & teacher provided n/w with answers. N/w can compare the answers "correct" ones are identified & corrections are made to it errors. (weights updation)

② Unsupervised learning:

- required when there is not a required data set with known answers.
- clustering is an example.
- dividing set of groups to
  say: Imagine searching of hidden pattern in a data set. An

- Clustering is an example for this. → dividing a set of elements into groups according to some unknown pattern.

③ Reinforcement learning :

- A strategy built on observation.

say: A little mouse running through a Maze. If it turns left cheese is found, if right then shock.

- With the course of time the mouse will learn to turn left.
- Its observations is negative(its NN makes a decision to turn (left/right) & observes (yum/ouch), the n/w can adjust its weights in order to made a diff decision the next

→ Application of NN :

1) Pattern Recognition (Based on previous - current DB)
2) Time series Prediction (predictive Modeling) ← (predictions)
3) Signal Processing (cochlear implants & hearing aids needs filtering of unnecessary noise & sound amplification)
4) control (self-driving cars etc)
5) Soft Sensors
6) Anomaly detection

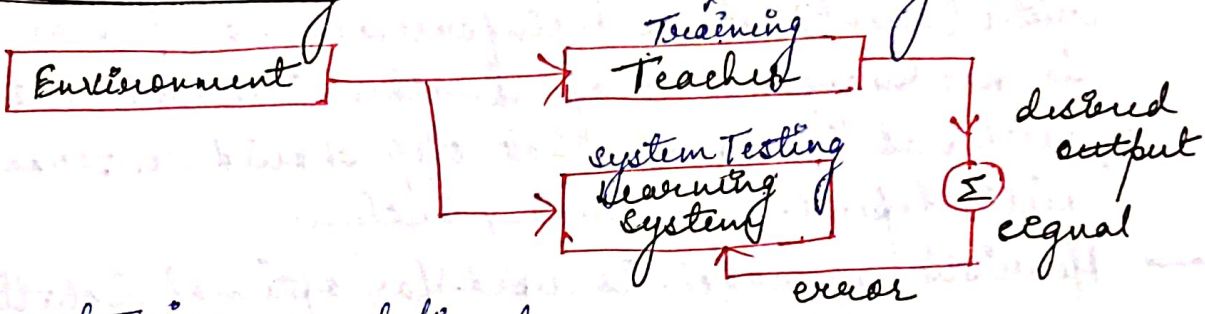
Say: Human & Monkey tested. It might happen that monkey in standing position may give a positive result as human. So in this case negative data is giving a positive result.

→ Still system should give correct judgement, such system is Invariant System.

→ Learning

① **Supervised Learning**: labelled data / learning with a teacher.

Eg:



→ categories are defined

A B Teacher (training)

If values passed via system either will be in A or B else not anywhere.

② **Unsupervised Learning**: unlabelled data / learning input a teacher

→ distance is calculated within similar objects & thus classification of data is done

Say: Manhattan distance, Bhattacharya distance, Hamming distance, Mahalanobis distance etc

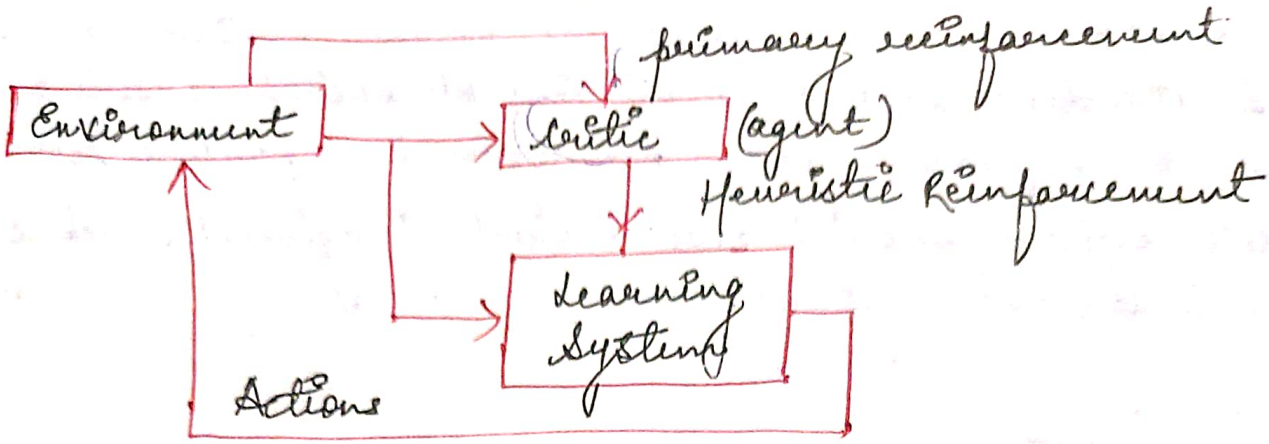
→ type of distance is selected depending upon model.

→ if distance ≈ 0 then yes similar else not.

③ **Reinfared learning**:

→ options will be given, we have to select the desired op.

→ a no of improvements but it will not be clear as of what feature is to be selected, this task will be done by the system.



→ it is also a kind of teacher learning but here critic (agent) acts as reinforcement. and here o/p is not mapped but of further improvement but will not tell as of what step should be taken it will depend on learning system

→ Heuristic approach is used for optimal solutions.

→ EDX (Online courses) MOOCS (Massive open online courses), swayam
Courseera, NPTEL

→ **PERCEPTRON**

→ invented in 1957, Frank Rosenblatt at Cornell Aeronautical Lab
→ this is simplest NN, single Neuron.

are sent into the neuron, are processed & result in an
→ neuron reads left to right info!

Step 1 Receive inputs

- the perceptron i/p are x1 and x2

input 0 ; x1 = 12

input 1 ; x2 = 4

Step 2 Weight inputs

- each i/p is to neuron must be weighted (multiplied by any value b/w (-1 to +1))

weight 0 ; 0.5

weight 1 ; -1

inputs multiplied by weights;

input 0 * weight 0 = 12 x 0.5 = 6

input 1 * weight 1 = 4 x (-1) = -4

Step 3 Sum inputs

Sum = 6 + (-4) = 2

Step 4 Generate output

→ the sum is generalized by passing the sum through the activation function

→ Activation function tells the neuron to 'fire' or 'not fire'.

→ Activation function get a little hairy.

→ if the output is positive → output = 1
if Negative → output = 0

→ activation function is sign of the sum (+/-)

→ The Perceptron Algorithm

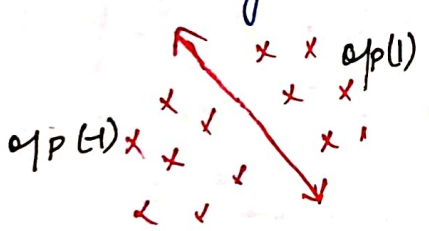
1) For every i/p, multiply its i/p by its weight
2) Sum of all the weighted inputs
3) Compare the o/p of the perceptron, based on the sum passed through activation function (sign (+/-) of the sum).

```
float[] inputs = {12, 4};
float[] weights = {0.5, -1};
float sum=0;
    for(int i=0; i< inputs.length; i++)
      { sum += inputs[i] * weight[i]; }
float output = activation(sum);
    int activation(float sum)
    {   if(sum > 0)
            return 1;
        else
            return -1;
    }
```

→ always i/p, weights

→ **Simple Pattern Recognition Using Perceptron:**

Say: A 2-D space. Points in that space can be classified as lying on either one side of the line or other.

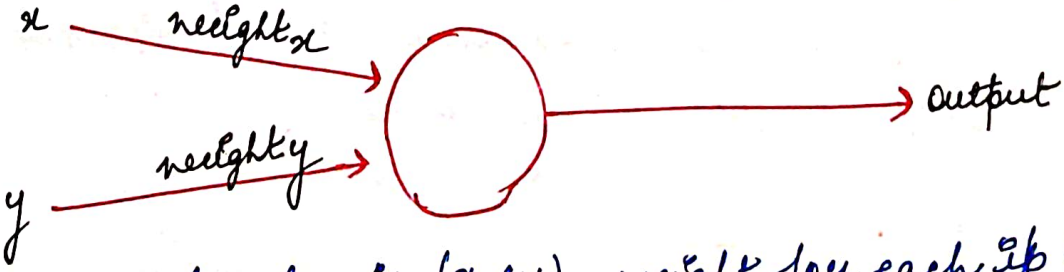


- say a space
  - cartesian (x, y)
  - polar (r, θ)

cartesian

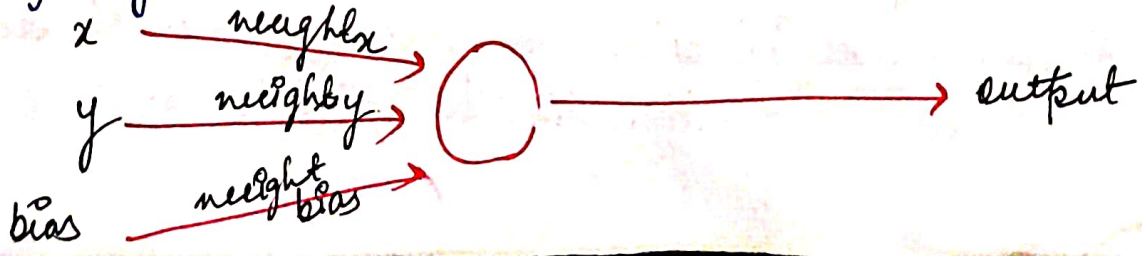
→ say a perceptron has 2 i/p (x-y coordinates). Using a sign activation function o/p will be either (+1/-1).

→ the i/p data is classified according to the sign of the o/p.



→ we see here two i/p (x & y) a weight for each i/p (weight x & y)

→ If x=y=0 then what.



→ If the i/p = 0 then where to place that. to avoid this dilemma.
→ we require third input typically referred to a Bias input.
→ Bias is always 1.0 & is also weighted.

0 * weight x = 0
0 * weight y = 0
1 * weight bias = weight bias

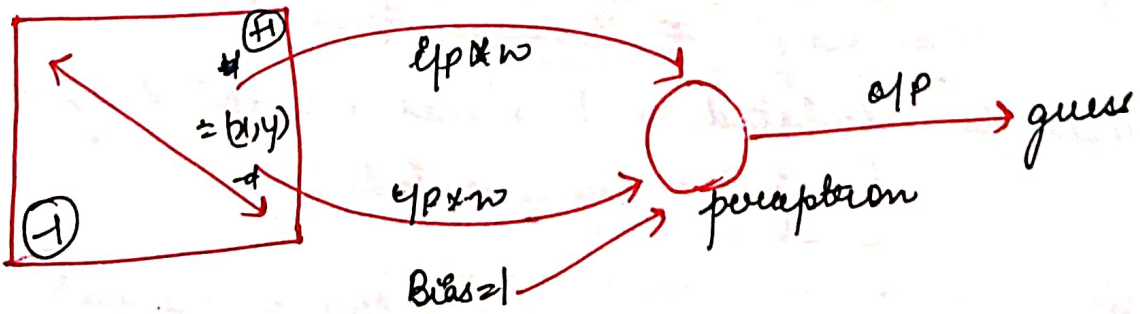
→ o/p is sum of all above three.
→ It biases the perceptron understanding of the computation relative to (0,0)

→ Coding the perceptron ((perceptron) coding in C)

```
class perceptron
{
  float [] = weights;
  perceptron (int, n)
  {
    weights = new float[n];

    weight = rand (-1,1);

  int feed_forward
```




```
perceptron p = new perceptron(3);
float[] point = {50,-12,1};
int result = p.feedforward(point);
```

January 6, 2018 → guess can be extended (beyond yes/No)

→ The way the nn can find out if it has made a correct guess. If its incorrect weights are adjusted. The process is:

1. provide the perceptron with i/p for which there is a known ans.
2. Ask the perceptron to guess the answer
3. Compute the error. (did it get the answer right or wrong)
4. Adjust the weights according to the error.
5. Return to step 1 and repeat.

→ Step 1 and 2 can be packaged into a function

error = desired output − guess output

→ perceptron only has two possible values as output +1 or −1. So there are only three possible errors.

→ if guess = desired output (perceptron guesses correct answer) is 0.

Desired	Guess	Error
−1	−1	0
−1	+1	−2
+1	−1	+2
+1	+1	0

→ Error is the determining factor of how weights are adjusted.

→ change in weight (Δ weight)

New weight = Δ weight + weight

→ Δ weight is calculated as the error multiplied by i/p

weight = Error * input

→ therefore, New weight = weight + Error * input

→ NN employes a strategy with a variable "Learning Rate"/Constant"

New weight = weight + Error * input * learning Constant

→ Logic & Logical operations:

① Logical AND → According to the truth table, we only want a neuron to produce o/p (1) when both i/ps are activated

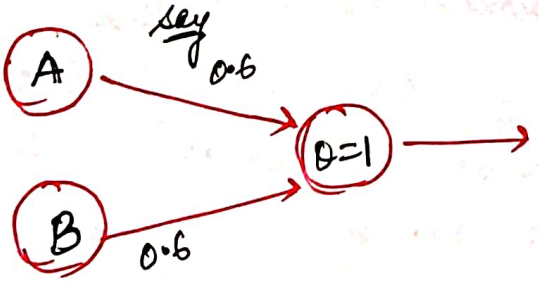
A	B	A∩B
0	0	0
0	1	0
1	0	0
1	1	1

→ Neuron Implementations: In 1945, scientist Mc Culloch Warren, & logician Walter Pitts. showed a paper showing how neurons implement 3 gates. The Neuron they used were simple threshold Neuron (Perceptions).

→ To do this we want sum of both the i/p greater than the threshold, but each i/p alone should be less than threshold

→ Lets use a threshold of (1) and represent it by θ, so now we need to choose the weights according to the constraints we have explained.

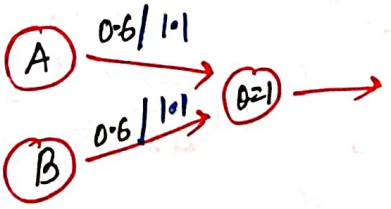
(equal & unequal weights)



$0.6 \times 0 + 0.6 \times 0 = 0 < 1 = 0$
$0.6 \times 0 + 0.6 \times 1 = 0.6 < 1 = 0$
$0.6 \times 1 + 0.6 \times 0 = 0.6 < 1 = 0$
$0.6 * 1 + 0.6 \times 1 = 1.2 > 1 = 1$

② OR

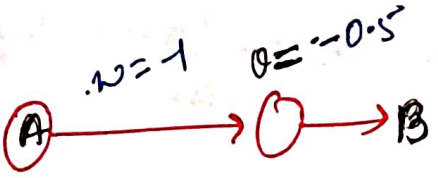
A	B	A∪B
0	0	0
0	1	1
1	0	1
1	1	1



$1.1 \times 0 + 1.1 \times 0 = 0 < 1 = 0$
$1.1 \times 0 + 1.1 \times 1 = 1.1 > 1 = 1$
$1.1 \times 1 + 1.1 \times 0 = 1.1 > 1 = 1$
$1.1 \times 1 + 1.1 \times 1 = 2.2 > 1 = 1$

③ NOT

A	~B
0	1
1	0

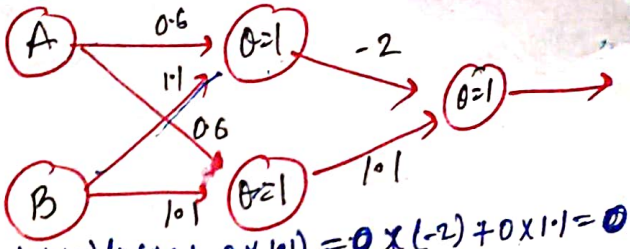


$-1 \times 0 = 0 > -0.5 = 1$
$-1 \times 0 = 0 > -0.5 = 0$

④ XOR Exclusive OR.

A	B	A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

3rd Jan 2016



I $(0\times0.6+0\times0.6)(0.6\times0+0\times1.1) = 0\times(-2)+0\times1.1 = 0$

II $0\times0.6+0$.

January 11, 2018

- This puts a pause in NN research for a long time coz it is not possible to create XOR with single Neuron or single layer → you need ② layers
- XOR means either table of 1 with A=1 or B=1 but not A=B=1.

$A\oplus B = A \cup B \wedge \sim(A \cap B)$

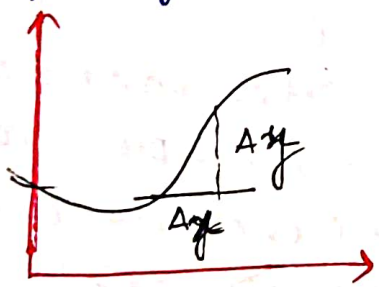
→ LEARNING RULE / ALGORITHM:

→ presented steps to learn the rule.

① gradient & derivatives ② gradient descent Minimization

→ diff calculus is branch of maths concerned with gradient (rate of change)

- the gradient/rate of change of $f(x)$ at a particular value of x,
- x approximated by $\Delta y/\Delta x$



$$\frac{\partial f(x)}{\partial x} = \lim_{\Delta x \to 0} \frac{\Delta y}{\Delta x} = \lim_{\Delta x \to 0} \frac{f(x+\Delta x) - f(x)}{\Delta x}$$

partial derivative of $f(x)$ w.r.t. x.

Say:

1) $f(x) = ax+b \Rightarrow \frac{\partial f(x)}{\partial x} \lim_{\Delta x \to 0} \frac{[a[x+\Delta x]+b] - [ax+b]}{\Delta x} = a$

2) $f(x) = a\cdot x^2 \Rightarrow \frac{\partial f(x)}{\partial x} \lim_{\Delta x \to 0} \frac{[a(x+\Delta x)^2] - [ax^2]}{\Delta x} = 2ax$

3) $f(x) = ax^n \Rightarrow n a x^{n-1}$

4) $f(x) = \log_e(x) = 1/x$

5) $f(x) = g(x) - h(x) \Rightarrow \lim_{\Delta x \to 0} \frac{[g(x+\Delta x) + h(x+\Delta x)) - g(x+\Delta x) + h(x)}{\Delta x}$ $\frac{\partial g(x)}{\partial x} \frac{\partial h(x)}{\partial x}$

6) $f(x) = e^{ax} \Rightarrow \frac{\partial f(x)}{\partial x} = a e^{ax}$

7) $f(x) = \sin(x) \Rightarrow \frac{\partial f(x)}{\partial x} = \cos(x)$

② Gradient Descent Minimization

→ If we have a function $f(x)$ & we want to change the value of x to minimize $f(x)$.

$\frac{\partial f}{\partial x} > 0$ → $f(x)$ increase as $x \uparrow$ so decrease x

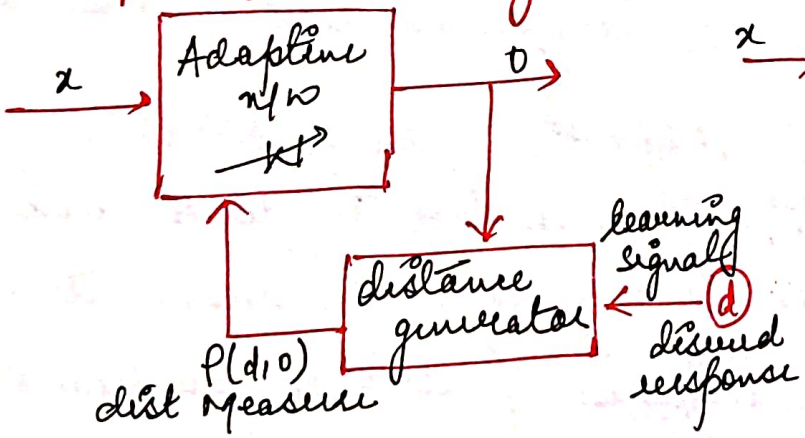
$\frac{\partial f}{\partial x} < 0$ → $f(x)$ decreases as $x \downarrow$ so increase x

$\frac{\partial f}{\partial x} = 0$ → $f(x)$ is at min/max then no change.

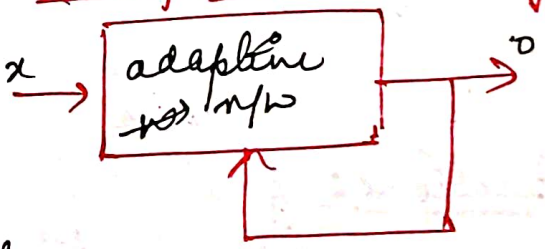
$$\Delta x = x_{new} - x_{old} = -\eta \frac{\partial f}{\partial x}$$

→ where η is small (+) constant specifying how much we change x by $\partial f/\partial x$. If we repeatedly use eq, $f(x) \to$ min call GDM.

→ Supervised learning



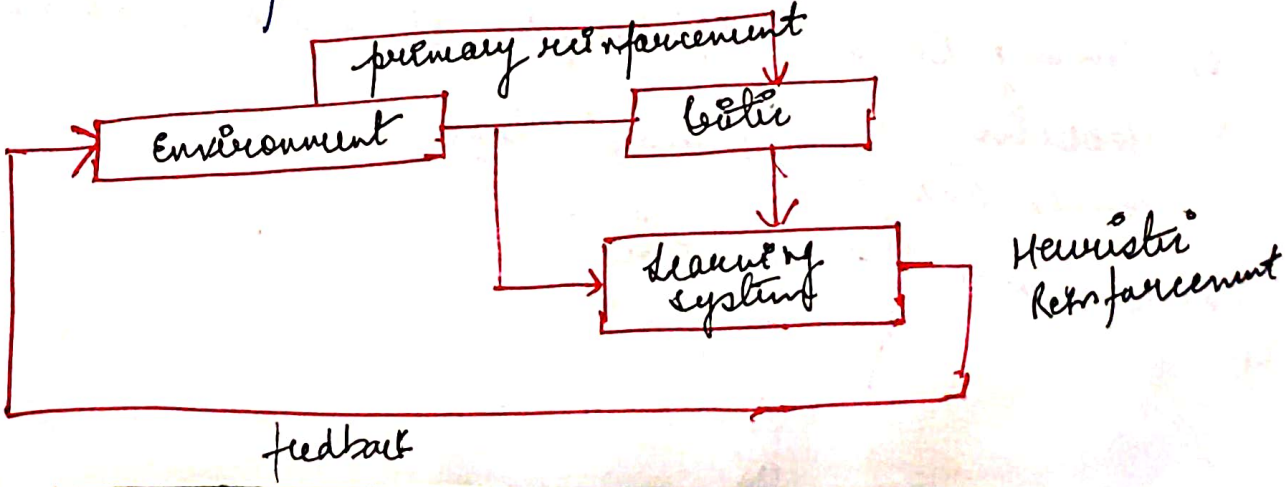
Unsupervised learning



→ REINFORCEMENT LEARNING: (again & again → repetation)

→ learning without teacher

→ No teacher to provide (desired) direct response at each step.



① Training Set: Symbols of I/p used to train the system
② Validation Set: the ensemble of sample that is used to validate the parameters during training.
③ Test Set: 'Input-desired' response data used to verify the performance to train the system. This data is not used for training.
④ Training Epoch: one cycle through the set of training patterns
⑤ Generalization: ability of NN to produce reasonable response to I/p patterns that are similar, not identical to training patterns.
⑥ Asynchronous: weights updated/activations at one time rather than simultaneously
⑦ Synchronous: weight/activation updated at same time.
⑧ Inhibitory Connection: Connection link b/w 2 neurons such that signal sent over this link will reduce the activation of the neuron that receives the signal.
⑨ Activations: A node's level of activity, the result of applying the activation f to the net I/p to the node. This is the value of the nodes transmits.

Yug Narsina
KK Aggarwal
January 12, 2018 → Learning is a process by which the free parameters of Mendel/McLaren 1970)

→ Learning Rules

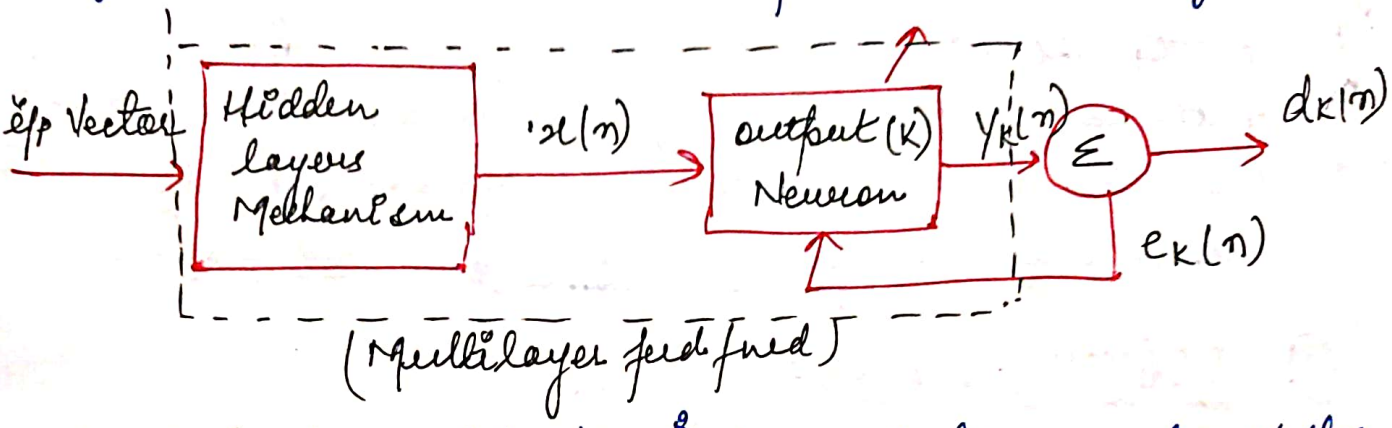
1) Error-Correction — optimum filtering
2) Memory-Based — Memorizing the training data explicitly
3) Hebbian — } Neurobiological
4) Competitive —
5) Boltzman — Statistical

① Error-Correction Learning:

→ error signal = desired response − output signal

→ $e_K(n) = d_K(n) - y_K(n)$

→ $e_K(n)$ actuates the control mechanism to make o/p signal $y_K(n)$ closer to the desired response $d_K(n)$ step by step.



→ Acost function $\mathcal{E}(n) = \frac{1}{2} e_K^2(n)$ is instantaneous value of the error energy → a steady state

→ (A delta rule or Widrow-Hoff rule)

$$\Delta w_{kj}(n) = \eta\, e_k(n)\, x_j(n)$$

(learning parameter)

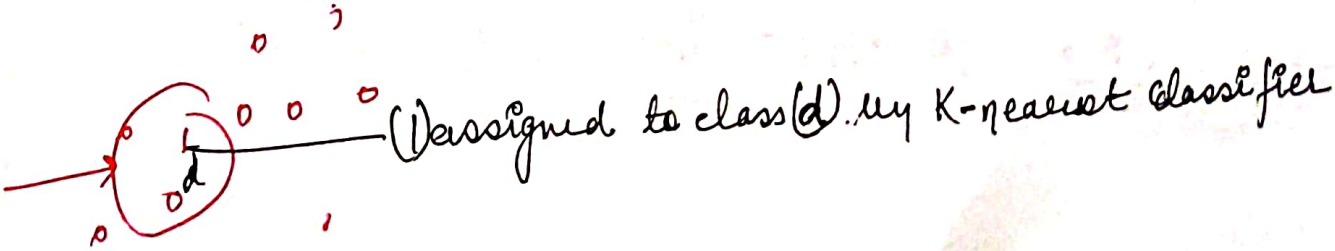
→ the adjustment is done in synaptic weight of a neuron proportional to the prod. of error & i/p signal of the synapse

$$w_{kj}(n+1) = w_{kj}(n) + \Delta w_{kj}(n)$$ → delta / gradient / change

② Memory-Based Learning:

→ all of the past experiences explicitly stored in a large memory of correctly classified i/p/o/p examples.

$$\{(x_i, d_i)\}_{i=1}^{N}$$



→ criterion used for defining the local neighbourhood of the test vector (x_{test}).
→ learning applied to the training eg. in local neigh! of x_{test}
→ Nearest

$$x_N \in \{x_1, x_2 \ldots x_N\}$$
$$d\{x_i, x_{test}\} = d\{x_N, x_{test}\}$$

→ if classified eg. $d(x_i, d_i)$ are independently & identically distributed.
→ If the sample N, is infinitely large (practically Not possible)
→ Classification error encountered by nearest nbr rule is bounded above twice the Bayes probability of error.

③ Hebbian Learning :

→ ① If two neurons on the other side of synapse are activated simultaneously then weight/strength of that synapse is selectively increased
② If two neuron on either side of synapse are activated asyn. then weights/strength of that synapse are selectively weakened/diminished
→ time dependent mech
local
interactive
conjunction & co-relation (previous result is used)
→ Hebbian synapse ↑ strength with (+) correlated pre/post synaptic & decreases when uncorrelated synaptic

→ $\boxed{\Delta w_{kj}(n) = F(y_k(n); x_j(n))}$ Hebbian Hypothesis
→ Simple form – $\boxed{\Delta w_{kj}(n) = \eta\, y_k(n)\, x_j(n)}$
→ Covariance Hypothesis – $\boxed{\Delta w_{kj}(n) = \eta\,(x_j - \bar{x})\,(y_j - \bar{y})}$

NOTE:-

① w_{kj} ↑↑ when $x_j > \bar{x}$ and $y_k > \bar{y}$ are both satisfied

② w_{kj} ↓↓ when $(x_j > \bar{x}$ and $y_k < \bar{y})$ or $(x_j < \bar{x}$ and $y_k > \bar{y})$

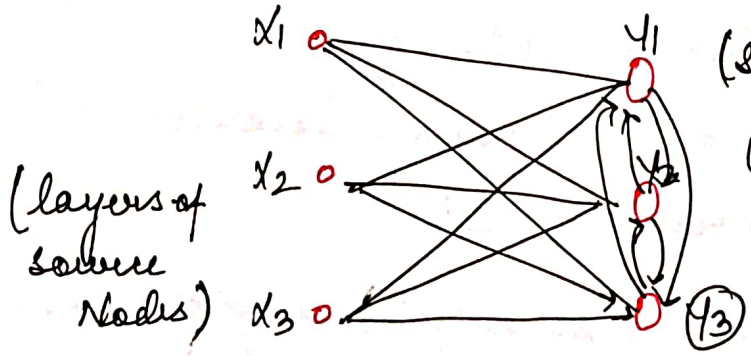
④ Competitive Learning:-

→ the output neurons of a NN complete among them are active.

→ a set of neurons that are all the same (except - weights)

→ a limit imposed on the strength of each neuron.

→ a mechanism that permits the neurons to complete → a winner takes all.



(layers of source Nodes)

(Single layer of o/p Neuron)

(all connections are present)

(feed forward)

→ $\boxed{\Delta w_{kj} = \eta \cdot (x_j - w_{kj})}$

NOTE:- all the neurons in the n/w are constrained to have the same length.

→ HEBB'S LEARNING RULE:

→ Given by Hebb in 1949.

→ If a neuron is influencing other neurons then weights are updated.

$$w_{i(new)} = w_i(old) + x_i \cdot d$$

$$b(new) = b(old) + d$$

Que ①

AND gate

x_1	x_2	d	$w_1=0$	$w_2=0$	$b=0$
1	1	1	1	1	1
1	-1	-1	0	2	0
-1	1	-1	1	1	-1
-1	-1	-1	2	2	-2

Sol ① $w_{1n} = w_{10} + x_1 \cdot d$ $\quad$ $w_2 = w_{20} + x_2 d$ $\quad$ $b = b_0 + d$

$\boxed{w_1 = 1}$ $\quad$ $\boxed{w_2 = 1}$ $\quad$ $\boxed{b = 1}$

② $w_{1n} = w_{10} + x_1 \cdot d$ $\quad$ $w_{2n} = w_{20} + x_2 \cdot d$ $\quad$ $b = b_0 + d$

$w_{1n} = 1 + 1 \cdot (-1)$ $\quad$ $w_{2n} = 1 + (-1)(-1)$ $\quad$ $b = 1 + (-1)$

$\boxed{w_1 = 0}$ $\quad$ $\boxed{w_{2n} = 2}$ $\quad$ $\boxed{b = 0}$

③ $w_{1n} = w_{10} + x_1 \cdot d$ $\quad$ $w_{2n} = w_{20} + x_2 \cdot d$ $\quad$ $b = b_0 + d$

$w_{1n} = 0 + (-1)(-1)$ $\quad$ $w_2 = 2 + (1)(-1)$ $\quad$ $b = 0 + (-1)$

$\boxed{w_{1n} = 1}$ $\quad$ $\boxed{w_2 = 1}$ $\quad$ $\boxed{b = -1}$

④ $w_{1n} = w_{10} + x_1 d$ $\quad$ $w_{2n} = w_2 + x_2 d$ $\quad$ $b = b_0 + d$

$w_1 = 1 + (-1)(-1)$ $\quad$ $w_{2n} = 1 + (-1)(-1)$ $\quad$ $b = -1 + 1$

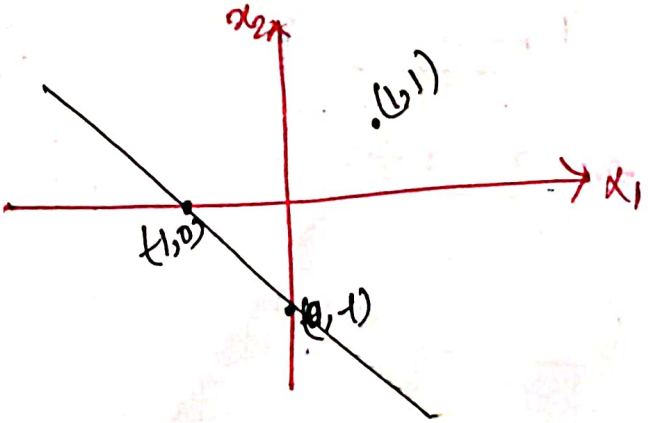
$\boxed{w_1 = -1 + 1 = 2}$ $\quad$ $\boxed{w_2 = 2}$ $\quad$ $\boxed{b = -2}$

I

$$x_2 = \frac{-w_1}{w_2} x_1 - \frac{b}{w_2}$$

$$= \frac{-1}{1} x_1 - \frac{-1}{1}$$

$$\boxed{x_2 = -x_1 - 1}$$

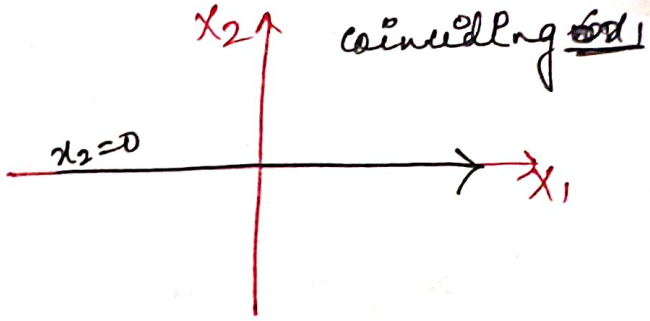


II

$$x_2 = -\frac{w_1}{w_2}x_2 - \frac{b}{w_2}$$

$$= \frac{-0}{2}x_2 - \frac{0}{2}$$

$$\boxed{x_2 = 0}$$

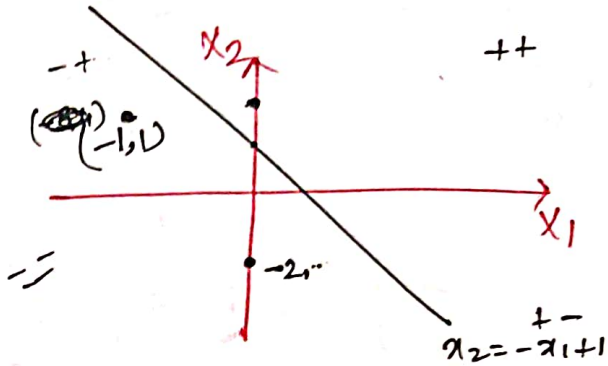


III

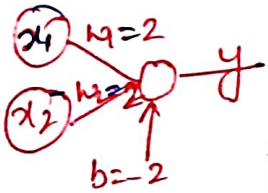
$$x_2 = -\frac{w_1}{w_2}x_2 - \frac{b}{w_2}$$

$$x_2 = -\frac{(-1)}{1}(x_1) - \frac{(-1)}{1}$$

$$\boxed{x_2 = +x_1 + 1}$$



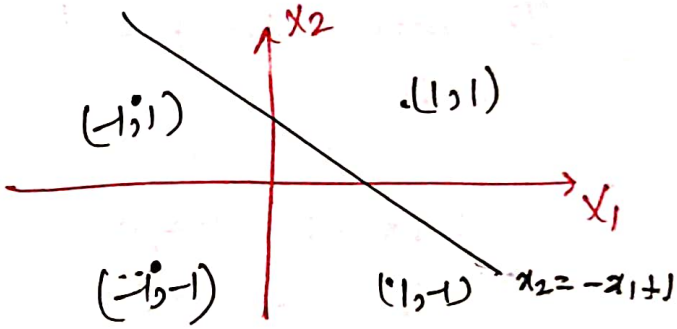
IV



$$x_2 = -\frac{w_1}{w_2}x_1 - \frac{b}{w_2}$$

$$= -\frac{2}{2}x_1 - \frac{(-2)}{2}$$

$$\boxed{x_2 = -x_1 + 1}$$



→ thus line $\boxed{x_2 = -x_1 + 1}$ linearly classifies i/p into (+/+).

Que 2 OR gate

x_1	x_2	d	$w_1=0$	$w_2=0$	$b=0$
1	1	1	1	1	1
1	-1	1	2	0	2
-1	1	1	1	1	3
-1	-1	-1	2	2	2

① $w_1 = 0 + 1(1)$ → $\boxed{w_1 = 1}$ $\quad w_2 = 0 + 1(1)$ → $\boxed{w_2 = 1}$ $\quad b = 0 + 1$ → $\boxed{b = 1}$

② $w_1 = 1 + 1(1)$ → $\boxed{w_1 = 2}$ $\quad w_2 = 1 + (-1)(1)$ → $\boxed{w_2 = 0}$ $\quad b = 1 + 1$ → $\boxed{b = 2}$

③ $w_1 = 2 + (-1)(1)$ → $\boxed{w_1 = 1}$ $\quad w_2 = 0 + (1)(1)$ → $\boxed{w_2 = 1}$ $\quad b = 2 + 1$ → $\boxed{b = 3}$

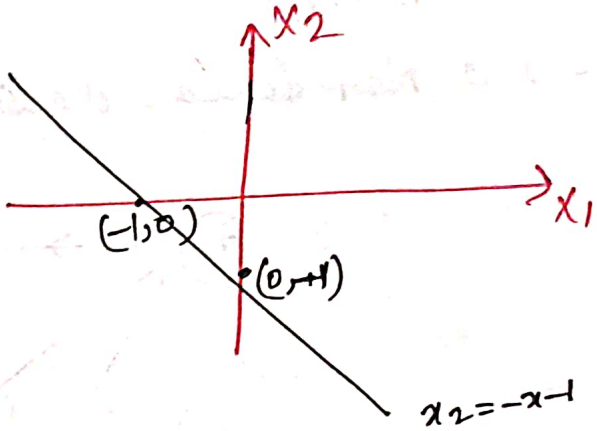
(4) $w_1 = 1 + (-1)(-1)$ $w_2 = 1 + (-1)(-1)$ $b = 3 + (-1)$

$\boxed{w_1 = 2}$ $\boxed{w_2 = 2}$ $\boxed{b = 2}$

I $x_2 = \frac{-w_1}{w_2} x_1 + \frac{b}{w_2}$

$= -\frac{1}{1} x_1 - \frac{1}{1}$

$\boxed{x_2 = -x_1 - 1}$



II $x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2}$

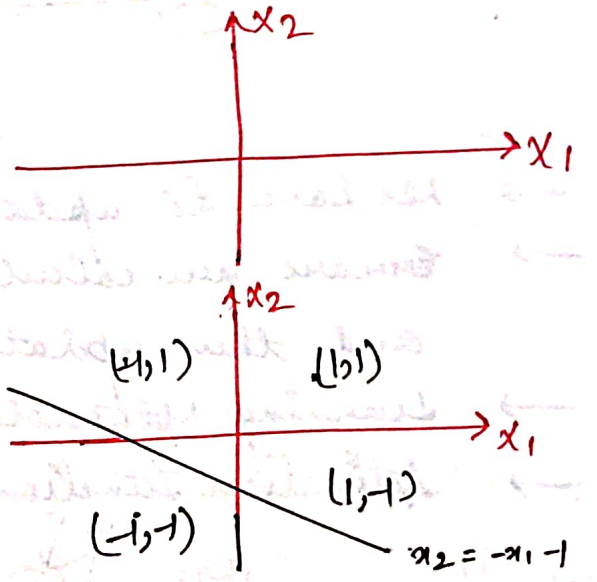
$\boxed{x_2 = \infty}$

III $x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2}$

$\boxed{x_2 = -x_1 - 3}$

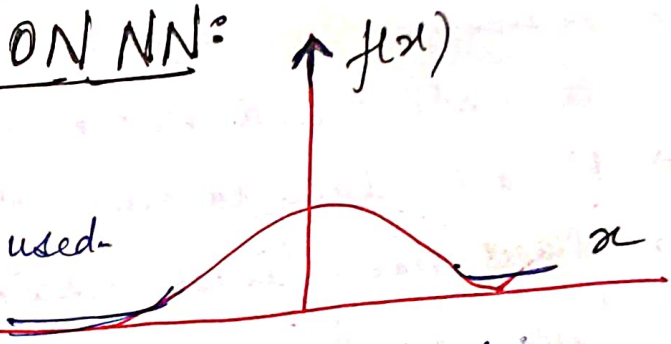
IV $x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2}$

$\boxed{x_2 = -x_1 - 1}$



→ the line $\boxed{x_2 = -x_1 - 1}$ linearly separates OR gate.

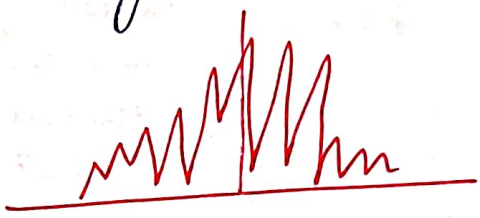
→ RADIAL BASIS FUNCTION NN:



→ uses Gaussian function

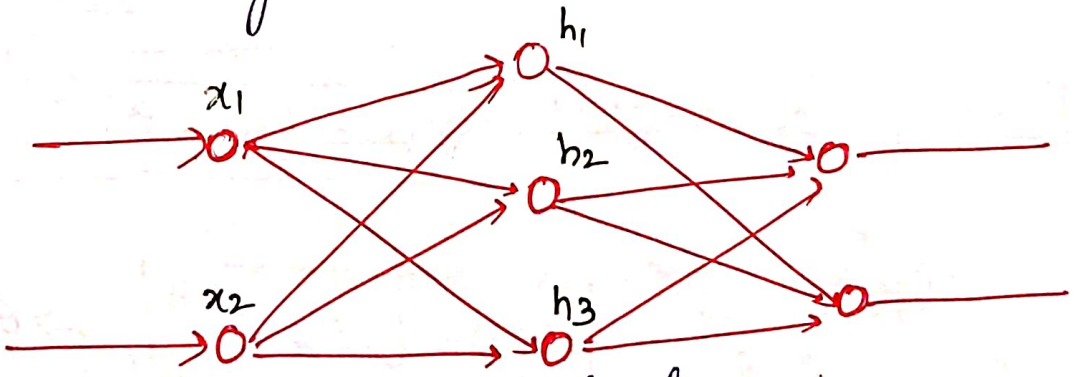
→ Natural (data) gaussian is used.

→ The data in computer is of gaussian function (to observe this plot the histogram) which might not exactly look like but partially look like Gaussian



September 7, 2017

→ Specific kind of Multi-layer perceptron (MLP) consisting of only 1 hidden layer.



→ Non-linear kind of basis function (gaussian fn) an asymptotic curve (Asymptotic curve → do not intersect axis and likely to intersect the axis at infinity) is used here

→ RBFNN → can be either supervised or unsupervised NN.

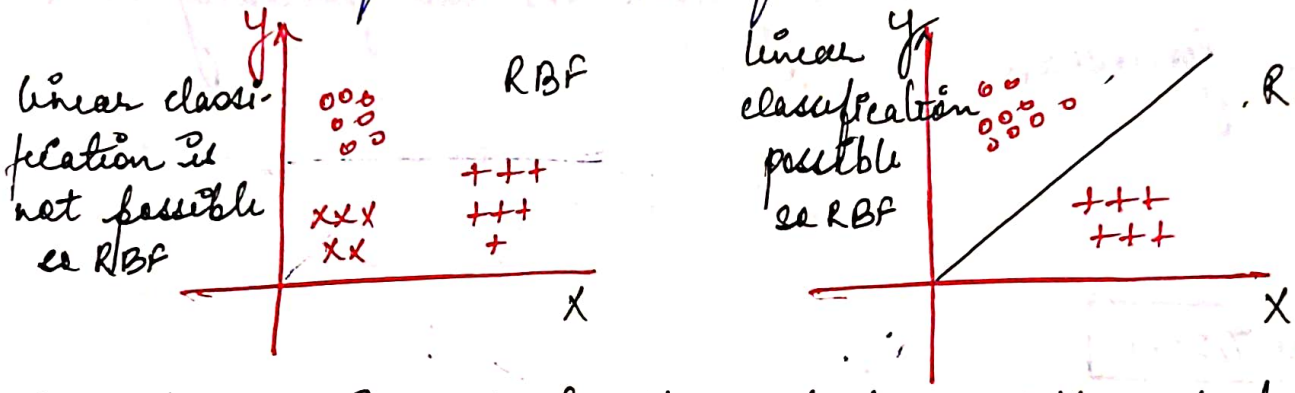
→ We have the predefined targets and deff i/p. Target and o/p is calculated and minimization is done using some delta learning or gradient (function) learning.

$$E = \frac{1}{2} \sum_{i=1}^{n} (t_i - O_i)^2$$ (gradient descent)

→ In unsupervised learning, we use clustering by defining the radius and width of the n/w. (μ, σ)

→ We also have to calculate centre of Radius.

→ place when linear classification is not applicable then we use non-linear classifier say (RBF). This RBF can also be used for linear classification also.



→ NN is a kind of classifier which uses different functions

① Gaussian function $\Phi(r) = \exp\left(\frac{-r^2}{2\sigma^2}\right)$, $\sigma > 0$

② Multi-Quadric function $\Phi(r) = (r^2 + \sigma^2)^{1/2}$, $\sigma > 0$

③ Generalized Multi-Quadric function $\Phi(r) = (r^2 + \sigma^2)^{\beta}$, $\sigma > 0$, $1 > \beta > 0$

④ Inverse Multi-Quadric function $\Phi(r) = (r^2 + \sigma^2)^{-1/2}$, $\sigma > 0$

⑤ Inverse generalized Multi-Quadric f^n $\Phi(r) = (r^2 + \sigma^2)^{-\alpha}$, $\sigma > 0$, $\alpha > 0$

⑥ Thin-plate spline f^n $\Phi(r) = r^2 \ln(r)$

natural log ($\log_e$)

⑦ Cubic function $\phi(r) = r^3$

⑧ Linear function $\phi(r) = r$

{'σ' → is the width, distance b/w clusters that can be made using any machine learning algo}

$$\sigma = \frac{d}{\sqrt{2h}}$$

d → distance b/w clusters (where clusters can be created)

h → max. no. of cluster points / centre

→ Comparing RBF and MLP

Differences

- RBF → Only 1 hidden layer
  MLP → multiple layers
- RBF → all are connected (every o/p is connected to o/p of hidden layer)
  MLP → might be fully or partially connected
- RBF → width & centre used.
  MLP → i/p & weight are combined
- RBF → both supervised & unsupervised learning
  MLP → only for supervised learning

Similarities

- for Non-linear classification
- universal approximation used in same application areas

- Single layer perceptron is for linear classification
- Multi layer perceptron is for Non-linear classification
- Activation forward diff = $\Delta x = x_1 - x_0$
- RBF is flexible in nature as it uses many fn.
- RBF based on interpolation
  ↓
  finding missing link by fixed then RBF (Activation forward)
- RBF → uses interpolation to estimate weights

→ Advantage : interpolation formula is easy to use.

→ Disadvantage : if data is noisy (not used for calculation) then interpolation results in faulty o/p as every i/p value is calculated in interpolation

→ as we are using formulae here so they should be programmed and thus will require more no. of line of codes so computation is inefficient

→ Assumptions (RBF improvements procedures) :

① No of hidden units < No of

② Change the way centres are defined (flexible defining of centres)

③ optimizing width parameters (σ)

④ introduce diff. choice of biasing

→ Gradient Descent Learning was used in Back propagation

$\frac{\partial E}{\partial x} > 0$ (decrease the x) eg $w_{s(n)} = w_s - \eta \frac{\partial E}{\partial w_s}$

$\frac{\partial E}{\partial x} < 0$ (add something to x) increase x

eg → $w_{s(n)} = w_s + \eta \frac{\partial E}{\partial w_s}$

$\frac{\partial E}{\partial x} = 0$ (No change) {Condition for obtaining (Maxima, Minima)}

→ (If η is not given assume it b/w 0 and 1 ($0 < \eta < 1$))

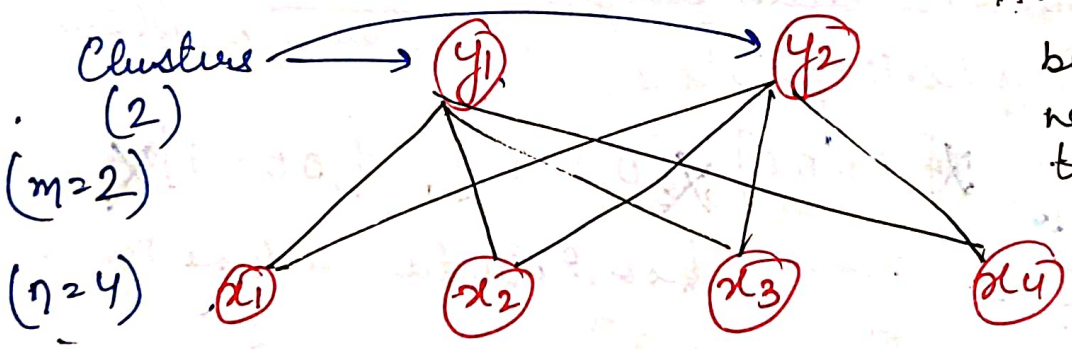
September 8, 2017 → UNSUPERVISED LEARNING

→ where targets are not given only data is given

→ Radial Basis f°. ex of super & unsup. learning

① SELF ORGANISING MAPS NN (SOM) NN

→ updation of weights but parameters change here
→ Euclidian distance calculated (b/w i/p and weights) (Sq. form)
→ Clusters are outputs (Mapped into pairs, say 100 o/p into 10 clusters, do 10 pairs of 10 o/p for 10 clusters)
→ Clustering is also a case of unsupervised learning.
→ Min-distance assigned to the cluster (Euclidian distance)
→ It organise its clusters itself so self-organising maps.
→ parameters are same as Supervised learning including one parameter (where no of cluster is defined) and learning rate is also an additional parameter.



Here y_1 and y_2 both have 4 weights correspond to each node

→ Characteristics of Self-Organizing Maps:-

① The inputs should map into corresponding clusters.
② No overlapping should be there.
③ Some of the data points may not be classified as if it does so (accuracy ≅ 100%) which is not possible. (Clustering algo with 100% not possible).

→ Parameters

① Input vectors
② Learning Rate
③ No of clusters

④ Euclidian distance in (square form)

$$D(j) = \sum_{i=1}^{n} \sum_{j=1}^{m} (x_i - w_{ij})^2$$

{ i = no of inp
{ j = no of cluster

→ The min value of D for j, we select it index val

⑤ D(J) ⇒ index for which D is minimum.

→ Updation of weights

Learning Rate

J fixed, i → vary

$$w_{ij(new)} = w_{ij(old)} + \alpha [x_i - w_{ij(old)}]$$

If J = 1 then

1.1
2.1
3.1
4.1

→ Such algorithm are called KOHONEN SOM.

⑦ VECTOR QUANTIZATION (VQ)

→ Labelling is done in supervised & clusters in Unsupervised.
→ VQ is supervised form of Unsupervised learning.
→ Class labelling - Clusters = No of updations
→ ① Class labelling = Cluster (T=1, C1) · addition updation
② Class labelling ~ Cluster (T=1, C2) subtraction updation

$T_x = C_j$	$w_{ij}(new) = w_{ij}(old) + \alpha [x_i - w_{ij}(old)]$
$T_x \neq C_j$	$w_{ij}(new) = w_{ij}(old) - \alpha [x_i - w_{ij}(old)]$

Learning VQ

→ VQ is used for data clustering based on Euclidean distance (D)

① Initially weight vector, Ip vector & class labels (T)

	T
x_1	1
x_2	2
x_3	2
x_4	1

C_1, C_2 — Clusters

② $i = 1 \to n$, $j = 1 \to m$ · $D(j) = \sum_{i=1}^{n} \sum_{j=1}^{m} (x_i - w_{ij})^2$

(3) Choose j where D is minimum

(4) $T_x = C_j$ or $T_x \neq C_j$ and update accordingly

(5) If $\alpha \downarrow\downarrow$ then (learning Rate reduced) → Accuracy ↑↑.

Que

Vectors	Class (T)
$X_1 = [0\ 0\ 0\ 1]$	2
$X_2 = [0\ 0\ 1\ 1]$	1
$X_3 = [1\ 0\ 0\ 0]$	2
$X_4 = [1\ 1\ 0\ 0]$	1
$X_5 = [0\ 1\ 1\ 0]$	1

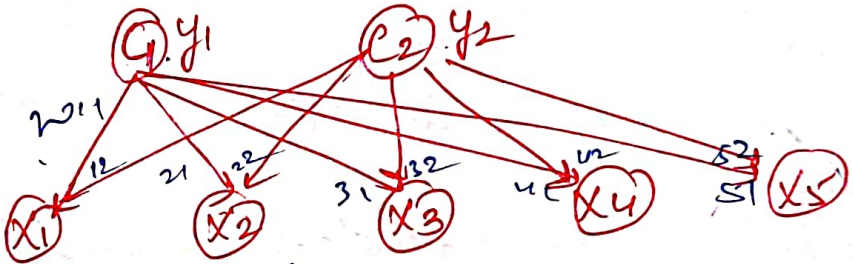
Solve acc to the no of iterations given in Que

$\alpha = 0.5$

$$W = \begin{matrix} y_1 & y_2 \end{matrix} \begin{bmatrix} 0 & 1 \\ 0 & 0 \\ 1 & 0 \\ 1 & 0 \end{bmatrix} \quad 4 \times 2$$

(X_1 consist of 4 elements i.e. why w consist of 4 elements)

Sol



(I) $D(j) = \sum_{i} \sum (x_i - w_{ij})^2$

(J=1) $= (x_1 - w_{11})^2 + (x_2 - w_{21})^2 + (x_3 - w_{31})^2 + (x_4 - w_{41})^2$

$= (0-0)^2 + (0-0)^2 + (0-1)^2 + (1-1)^2 \Rightarrow 0+0+1+0$

$D(1) = 1$

$D(2) = \sum_{i=1}^{4} \sum_{j=1}^{2} (x_i - w_{ij})^2$

$= (x_1 - w_{12})^2 + (x_2 - w_{22})^2 + (x_3 - w_{32})^2 + (x_4 - w_{42})^2$

$= (0-1)^2 + (0-0)^2 + (0-0)^2 + (1-0)^2 \Rightarrow 1+0+0+1$

$D(2) = 2$

→ Here D(1) is minimum so update (y_1) weights

(yaha pas ahyan dena Hai so)

Here, $D(1) \rightarrow T = 2$

$D(1) = C_1$ so $(C_1 = 1) \neq T = 2$

$T \neq C \Rightarrow w_{ij}(new) = w_{ij}(old) - \alpha [x_i - w_{ij}(old)]$

$w_{11} = 0 - 0.5(0-0)$ | $w_{21} = 0 - 0.5(0-0)$ | $w_{31} = 0 - 0.5(1-0)$ | $w_{41} = 1 - 0.5(1-0)$

$w_{11} = 0$ | $w_{21} = 0$ | $w_{31} = 0.5$ | $w_{41} = 1$

Updating weight vector,

$$w = \begin{bmatrix} 0 & 1 \\ 0 & 0 \\ 0.5 & 0 \\ 1 & 0 \end{bmatrix}_{4\times 2}$$

(II) $x_2 = [0\ 0\ 1\ 1]$

$$D(j) = \sum_{i}^{4} \sum_{j}^{2} (x_i - w_{ij})^2$$

J=1 $D(1) = (x_1 - w_{11})^2 + (x_2 - w_{21})^2 + (x_3 - w_{31})^2 + (w_{41} - x_{41})^2$

$D(1) = (0-0)^2 + (0-0)^2 + (1-1.5)^2 + (1-1)^2$

$\boxed{D(1) = 0.25}$

$D(2) = (x_1 - w_{12})^2 + (x_2 - w_{22})^2 + (x_3 - w_{32})^2 + (x_4 - w_{42})^2$

$= (0-1)^2 + (0-0)^2 + (1-0)^2 + (1-0)^2$

$\boxed{D(2) = 3}$

D(1) is minimum, $C(1) = T = 1$, $C = T$

updating,

$w_{11n} = 0 + 0.5(0-0)$ | $w_{21n} = 0 + 0.5(0-0)$ | $w_{31n} = 1.5 + 0.5(1-1.5)$ | $w_{41n} = 1 + 0.5(1-1)$

$\boxed{w_{11n} = 0.5}$ | $\boxed{w_{21n} = 0.5}$ | $\boxed{w_{31n} = 2}$ | $\boxed{w_{41} = 1}$

update weight vector,

$$w = \begin{bmatrix} 0.5 & 1 \\ 0.5 & 0 \\ 2 & 0 \\ 1 & 0 \end{bmatrix}_{4\times 2}$$

(III) $x_3 = [1\ 0\ 0\ 0]$

$D(1) = (1-0.5)^2 + (0-0.5)^2 + (0-2)^2 + (0-0)^2$

$= 0.25 + 0.25 + 4$

$\boxed{D(1) = 4.50}$

$D(2) = (1-0)^2 + (0-0)^2 + (0-0)^2 + (0-0)^2$

$\boxed{D(2) = 0}$

D(2) is min $\Rightarrow J = C_2$ & $T = 2$ $C = T$

$w_{11n} = 1 + 0.5(1-1)$ | $w_{21n} = 0 + 0.5(0-0)$ | $w_{31n} = 0 + 0.5(0-0)$ | $w_{41} = 0 + 0.5(0-0)$

$\boxed{w_{11n} = 1}$ | $\boxed{w_{21n} = 0}$ | $\boxed{w_{31n} = 0}$ | $\boxed{w_{41n} = 0}$

Updating weight Vector, $w = \begin{bmatrix} 0.5 & 1 \\ 0.5 & 0 \\ 2 & 0 \\ 1 & 0 \end{bmatrix}_{4\times 2}$

$x_q = [1\ 1\ 0\ 0]$

$D(1) = (x_1 - w_{11})^2 + (x_2 - w_{21})^2 + (x_3 - w_{31})^2 + (x_4 - w_{41})^2$

$D(1) = 0.25 + 0.25 + 4 + 1 \Rightarrow 5.5 \Rightarrow$ **$D(2) = 5.5$**

$D(2) = 0 + 1 + 0 + 0 \Rightarrow$ **$D(2) = 1$**

$D(2) = 1,\ T = 1,\ \ell = T$

$w_{12} = 0 + 1 + 0.5(1-1)$ **$w_{12} = 1$** | $w_{22} = 0 + 0.5(1-0)$ **$w_{22} = 0.5$** | $w_{32} = 0 + 0.5(0-0)$ **$w_{32} = 0$** | $w_{42} = 0 + 0.5(0-0)$ **$w_{42} = 0$**

updating the weight vector $w = \begin{bmatrix} 0.5 & 1 \\ 0.5 & 0.5 \\ 2 & 0 \\ 1 & 0 \end{bmatrix}$

September 14, 2017

(3) BOLTZMAN MACHINE & HOPFIELD NN:

→ They are probabilistic and Stochastic Model.

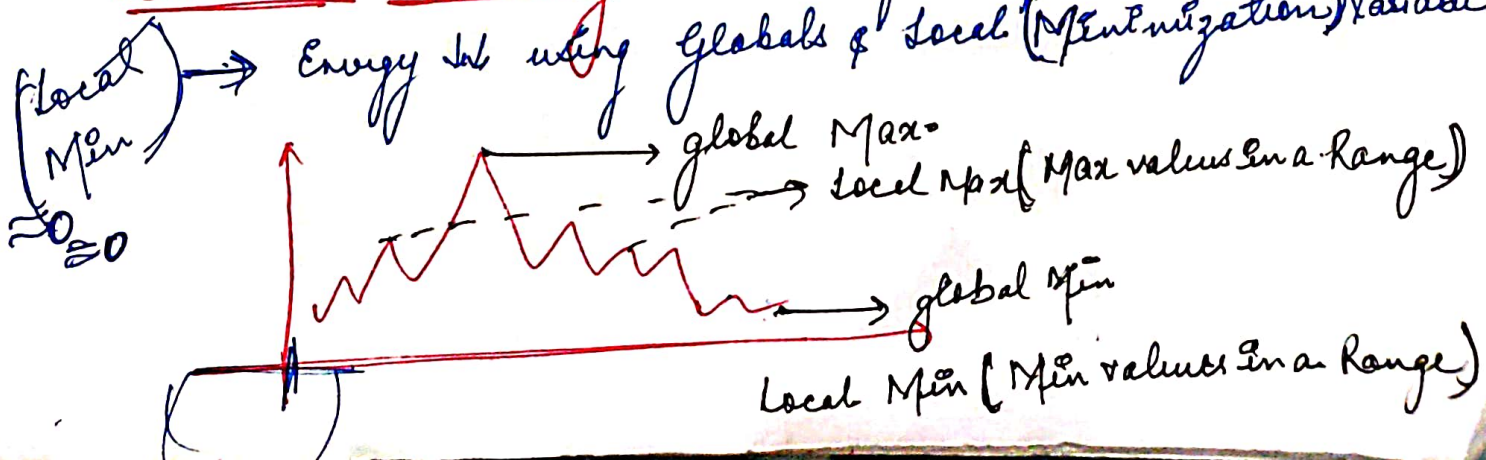
→ All the computations will be based on probability Model of Visible & Hidden layers.

→ probability distribution Models

→ Energy minimization (Here energy is comb of w, y_p & b) and then weights are updated.

- Simulated Annealing: Used to find optimal solution

(local Min) → Energy is using Globals & local (Minimization) Variable



→ all possible ways to find the solution is Search Space.

→ Search Space should be minimised.

→ ① Hill Climbing: finding sol' if not appropriate → Next → Next

② Heuristics: estimating the results.

→ Thermal Equilibrium (Energy) in AI.